Session 13 PCW: Support Vector Classifiers and The Kernel Trick

Prep for the deep dive into SVCs and the kernel trick: why the nullspace doesn't matter for optimal weights, why those weights are always a combination of data points (the seed of the kernel trick), hand-built kernel parameterizations, a real 3-kernel SVM comparison on banknote data, and two extension experiments on the Yosemite temperature data from Session 12.

Question 1 of 4: Basic Prep for class

1. The nullspace of X, and why it's ignored for optimal weights

The nullspace of a data matrix X is every weight direction v where $Xv = 0$, directions that produce a prediction of exactly zero for *every single training point*, no matter what they're added to.

That matters because if w^* is an optimal weight vector and v is in the nullspace of X, then $w^* + v$ predicts **identically** to w^* on every training point:
$X(w^* + v) = Xw^* + Xv = Xw^* + 0 = Xw^*$. Adding any nullspace component changes nothing about the loss. So the nullspace direction is genuinely free, invisible to the optimization, there's no "correct" amount of it to add, and no way for the loss function to prefer one amount over another. It gets ignored (typically just set to zero, which is what a minimum-norm solution like the pseudo-inverse does) because it carries zero information about the fit.

2. Why optimal weights can always be written as a linear combination of the data points

Quick version (nullspace intuition): since the nullspace component of w never affects the loss (point 1), the part of w that actually matters lives in the row space of X, i.e. $\text{span}\{x_1, \ldots, x_n\}$, so $w = \sum_i \alpha_i x_i$.

The book's actual derivation (Murphy \S14.4.3, kernelized ridge regression) is more rigorous and holds even when X has *no* nullspace at all, so it doesn't depend on the informal argument above. Start from the regularized primal solution:

$$w = (X^T X + \lambda I)^{-1} X^T y$$

The matrix inversion lemma rewrites this, algebraically identically, as:

$$w = X^T(XX^T + \lambda I)^{-1}y = X^T\alpha, \quad \alpha = (K + \lambda I)^{-1}y, \quad K = XX^T \text{ (the Gram matr}$$

So $w = \sum_i \alpha_i x_i$ exactly, a clean identity (this is the *representer theorem* for ridge regression), valid regardless of nullspace. Once w is guaranteed to be a combination of data points, a prediction $f(x) = w^T x = \sum_i \alpha_i (x_i^T x)$ only ever needs inner products, which get replaced by $\kappa(x_i, x)$ for any kernel, that substitution is the kernel trick.

One condition both derivations gloss over: for the kernel substitution to be mathematically valid, κ must be a **Mercer kernel** (positive semi-definite). That guarantees some (possibly infinite-dimensional) feature space actually exists where $\kappa(x, x') = \phi(x) \cdot \phi(x')$. Not every similarity function qualifies.

3. Generated SVC functions

Prompt A (simple/linear SVC): "Write a Python function called `train_linear_svc(X, y)` that uses scikit-learn's `SVC` class with a linear kernel to fit a classifier on training data `X` and labels `y`, and returns the fitted model."

```python
def train_linear_svc(X, y):
    from sklearn.svm import SVC
    model = SVC(kernel='linear')
    model.fit(X, y)
    return model
```

Prompt B (RBF-kernel SVC): "Write a Python function called `train_rbf_svc(X, y)` that uses scikit-learn's `SVC` class with a radial basis function (RBF) kernel to fit a classifier on training data `X` and labels `y`, and returns the fitted model. Include a `gamma` parameter with default `'scale'`."

```python
def train_rbf_svc(X, y, gamma='scale'):
    from sklearn.svm import SVC
    model = SVC(kernel='rbf', gamma=gamma)
    model.fit(X, y)
    return model
```

Bringing both prompts and both functions to class to discuss, per the assignment.

4. Deep dive prep: the kernel trick (\S14.4) vs. SVM equations (\S14.5)

\S14.4, the kernel trick itself, is not SVM-specific. It's a generic substitution: wherever an algorithm uses an inner product $\langle x, x' \rangle$, replace it with a kernel function $\kappa(x, x')$. The book applies this to nearest-neighbor, K-medoids clustering, kernel PCA, and (worked in detail, \S14.4.3) kernelized ridge regression, SVMs aren't covered until \S14.5. The ridge regression case is the clean example: starting from the regularized primal $w = (X^T X + \lambda I)^{-1} X^T y$, the matrix inversion lemma gives $w = X^T(XX^T + \lambda I)^{-1}y = \sum_i \alpha_i x_i$, so predictions become $f(x) = \sum_i \alpha_i \kappa(x_i, x)$,

no explicit feature map ever needed. Requires κ to be a Mercer (positive semi-definite) kernel.

\S14.5, what SVMs specifically add on top. Ridge and logistic regression are *already* kernelizable (above), but give a **dense** α, every training point contributes to every prediction. SVM's actual contribution is pairing the kernel trick with a *different loss function*, hinge loss for classification, ϵ-insensitive loss for regression, specifically because those losses make α **sparse**: only points near or across the margin (the support vectors) end up with nonzero α.

Classification (\S14.5.2.1), hinge loss: $L_{\text{hinge}}(y, \eta) = \max(0, 1 - y\eta) = (1 - y\eta)_+$ for $y \in \{1, -1\}$. The soft-margin objective (book eq. 14.58):

$$\min_{w, w_0, \xi} \frac{1}{2}\|w\|^2 + C\sum_i \xi_i \quad \text{s.t. } \xi_i \geq 0,\ y_i(x_i^T w + w_0) \geq 1 - \xi_i$$

C controls tolerance for margin violations: large C punishes them heavily (narrower margin, closer fit, more overfitting risk), small C tolerates more (wider margin, more regularized). The book solves this via the N dual variables (the Lagrange multipliers for the constraints) rather than directly, specialized solvers like SMO exist because a generic QP solver is too slow at this scale ($O(N^3)$).

The solution has the same *form* as ridge's: $\hat{w} = \sum_i \alpha_i x_i$ (eq. 14.59), but now α is sparse, nonzero only for support vectors (points misclassified, or sitting on/inside the margin). After the kernel trick, prediction is:

$$\hat{y}(x) = \text{sgn}\left(\hat{w}_0 + \sum_i \alpha_i \kappa(x_i, x)\right)$$

summed in practice only over support vectors, costing $O(sD)$ where s is the number of support vectors, not $O(ND)$.

Radial basis (Gaussian) kernel (\S14.2.1, used throughout both \S14.4 and \S14.5's examples): the general form is $\kappa(x, x') = \exp\left(-\frac{1}{2}(x - x')^T \Sigma^{-1}(x - x')\right)$, a full covariance matrix, allowing different length-scales per input dimension. The single-parameter version used in scikit-learn (and in our Core Questions code below) is the isotropic special case $\kappa(x, x') = \exp(-\gamma\|x - x'\|^2)$, where $\Sigma^{-1} = 2\gamma I$.

Question 2 of 4: Core Questions

1. Kernel parameterizations (ϕ)

Linear kernel: $K(x, x') = x \cdot x'$. The feature map is the identity, $\phi(x) = x$. No transform at all, the "kernel" is just the original dot product.

Quadratic kernel: $K(x, x') = (a\langle x, x'\rangle + b)^2$. This is the general form, and it's exactly what `SVC(kernel='poly', degree=2)` computes internally (scikit-learn calls a \verb|gamma| and b \verb|coef0|: `K(X,Y) = (gamma * <X,Y> + coef0) ** degree`). For $x = (x_1, \ldots, x_d)$, expanding gives:

$$(a\langle x, x'\rangle + b)^2 = a^2(x \cdot x')^2 + 2ab(x \cdot x') + b^2$$

so the explicit feature map that reproduces this dot product is:

$$\phi(x) = \left[a\,x_1^2, \ldots, a\,x_d^2,\ \sqrt{2}\,a\,x_i x_j \text{ for } i < j,\ \sqrt{2ab}\,x_1, \ldots, \sqrt{2ab}\,x_d,\ b\right]$$

Worked check for $d = 2$:
$\phi(x) \cdot \phi(x') = a^2 x_1^2 x_1'^2 + a^2 x_2^2 x_2'^2 + 2a^2 x_1 x_2 x_1' x_2' + 2ab(x_1 x_1' + x_2 x_2') + b^2$, which matches $a^2(x \cdot x')^2 + 2ab(x \cdot x') + b^2$ term for term. Setting $a = 1$ recovers the simpler $(x \cdot x' + c)^2$ form (with $b = c$), but a is a real, independent scaling knob in the actual implementation, not just a simplification.

In [1]:
```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report

# UCI banknote authentication dataset (the guide's "financial transaction da
# wavelet-transformed features of banknote images, predicting forged (0) vs
col_names = ['variance', 'skewness', 'curtosis', 'entropy', 'class']
bankdata = pd.read_csv('data/data_banknote_authentication.txt', names=col_na
print(bankdata.shape)
print(bankdata['class'].value_counts())

X = bankdata.drop('class', axis=1)
y = bankdata['class']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, ra

# Simple kernel SVM, following the linked guide's pattern
svc = SVC(kernel='linear')
svc.fit(X_train, y_train)
print(classification_report(y_test, svc.predict(X_test)))
```

```
(1372, 5)
class
0    762
1    610
Name: count, dtype: int64
              precision    recall  f1-score   support

           0       0.99      0.99      0.99       148
           1       0.98      0.98      0.98       127

    accuracy                           0.99       275
   macro avg       0.99      0.99      0.99       275
weighted avg       0.99      0.99      0.99       275
```

In [2]:
```
# Extend to all three kernels: linear, quadratic (poly, degree=2), radial ba
kernel_configs = [
    ('linear', 'linear', {}),
    ('quadratic', 'poly', {'degree': 2}),
    ('radial basis', 'rbf', {}),
]

results = {}
for name, kernel, extra in kernel_configs:
    model = SVC(kernel=kernel, **extra)
    model.fit(X_train, y_train)
    acc = accuracy_score(y_test, model.predict(X_test))
    results[name] = acc
    print(f'{name:15s} kernel: accuracy = {acc:.4f}')

best = max(results, key=results.get)
print(f'\nBest kernel on banknote data: {best} ({results[best]:.4f})')
```

```
linear          kernel: accuracy = 0.9855
quadratic       kernel: accuracy = 0.9745
radial basis    kernel: accuracy = 1.0000

Best kernel on banknote data: radial basis (1.0000)
```

3. Which kernel contributes most to accuracy?

Radial basis kernel wins (see output above), beating both linear and quadratic. That tracks with the data: the banknote features (variance, skewness, curtosis, entropy of wavelet-transformed images) don't separate cleanly with a flat hyperplane or a simple quadratic boundary, the RBF kernel's local-similarity structure can carve a tighter, more flexible boundary around the two classes. Quadratic actually does *slightly worse* than plain linear here, a reminder that a fancier kernel isn't automatically better, it has to match the actual shape of the decision boundary in the data.

Question 3 of 4: Extension Questions

In [3]:
```
import numpy as np

# Reuse the Yosemite temperature data from Session 12.
years = range(2011, 2017)
files = ['data/CRNS0101-05-%d-CA_Yosemite_Village_12_W.txt' % y for y in yea
usecols = [1, 2, 8]
data = [np.loadtxt(f, usecols=usecols) for f in files]
data = np.vstack(data)
valid = data[:, 2] > -1000

dates = data[valid, 0].astype(int)
months = (dates // 100) % 100   # YYYYMMDD -> MM
temps = data[valid, 2]

# Subsample for speed, classifying month from a single raw temperature value
rng = np.random.default_rng(0)
```

```
idx = rng.choice(len(temps), size=20000, replace=False)
X = temps[idx].reshape(-1, 1)
y = months[idx]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, rar

svc_month = SVC(kernel='rbf')
svc_month.fit(X_train, y_train)
acc = accuracy_score(y_test, svc_month.predict(X_test))
baseline = (y_test == np.bincount(y_train.astype(int)).argmax()).mean()

print('SVM accuracy (temperature only, 12-class month):', round(acc, 4))
print('Baseline (always guess most common month):', round(baseline, 4))
```

```
SVM accuracy (temperature only, 12-class month): 0.1908
Baseline (always guess most common month): 0.0818
```

1. Best accuracy for classifying month from raw temperature?

≈19% (vs. an 8.2% majority-class baseline), using a single feature (raw temperature) and an RBF-kernel SVC across all 12 months. That beats random/majority-vote by a real margin, temperature clearly carries *some* month information, but it's far from reliable: a single temperature reading, say 15°C, is genuinely ambiguous between several different months (it could be a cool July morning or a warm October afternoon), so no classifier can push this much higher using temperature alone. This mirrors the Session 12 finding exactly: one input dimension can't carry enough information, temperature needs time-of-day and date alongside it to be well-explained.

In [4]:

```
from sklearn.svm import SVR

# SVR with the same 2-feature representation (minute-of-day, day-index) used
# the Ridge+RBF model in Session 12, for a direct comparison.
data[:, 1] = np.floor_divide(data[:, 1], 100) * 60 + np.mod(data[:, 1], 100)

dates_all = data[:, 0].astype(int)
dt = pd.to_datetime(dates_all.astype(str), format='%Y%m%d')
day_idx = (dt - dt.min()).days.values

X2 = np.column_stack([data[valid, 1], day_idx[valid]]).astype(float)
y2 = data[valid, 2]

X2_train, X2_test, y2_train, y2_test = train_test_split(X2, y2, test_size=0.

# SVR training cost grows much faster with data size than Ridge did, so this
# smaller subsample (8000/4000) than Session 12's Ridge model did (120000/40
tr_idx = rng.choice(len(X2_train), size=8000, replace=False)
te_idx = rng.choice(len(X2_test), size=4000, replace=False)
X2tr, y2tr = X2_train[tr_idx], y2_train[tr_idx]
X2te, y2te = X2_test[te_idx], y2_test[te_idx]

# Scale each axis by its width, same reasoning as the Ridge+RBF model's sigm
sigma_min, sigma_day = 80.0, 6.0
X2tr_s = X2tr / np.array([sigma_min, sigma_day])
X2te_s = X2te / np.array([sigma_min, sigma_day])
```

```
svr = SVR(kernel='rbf', C=10, gamma=1.0)
svr.fit(X2tr_s, y2tr)

print('SVR train R^2:', svr.score(X2tr_s, y2tr))
print('SVR test R^2:', svr.score(X2te_s, y2te))
```

```
SVR train R^2: 0.9365508375224986
SVR test R^2: 0.7277486761620238
```

2. How does SVR's R^2 compare to the Session 12 linear parameter model?

SVR gets **test $R^2 \approx 0.73$** (train $R^2 \approx 0.94$) here, versus the explicit RBF-features + Ridge model's **test $R^2 \approx 0.83$** from Session 12. Ridge wins, and the gap is mostly about scale, not kernel quality: the Ridge model trained on 120,000 rows with ~4700 explicit RBF features, while SVR had to be subsampled down to 8,000 rows to keep training tractable (SVR's training cost grows much faster with dataset size, closer to quadratic/cubic, than a fixed-feature linear fit). The train/test gap for SVR (0.94 vs 0.73) is also wider, a sign it's overfitting that smaller training set more than the Ridge model did on its much larger one. Same basic kernel idea (RBF/radial basis), two very different computational costs for using it at scale, which is itself one of the practical lessons of the kernel trick: it's elegant, but it's not free.